

AI Security 5.1. RAG Security Fundamentals

Task 1 - Introduction

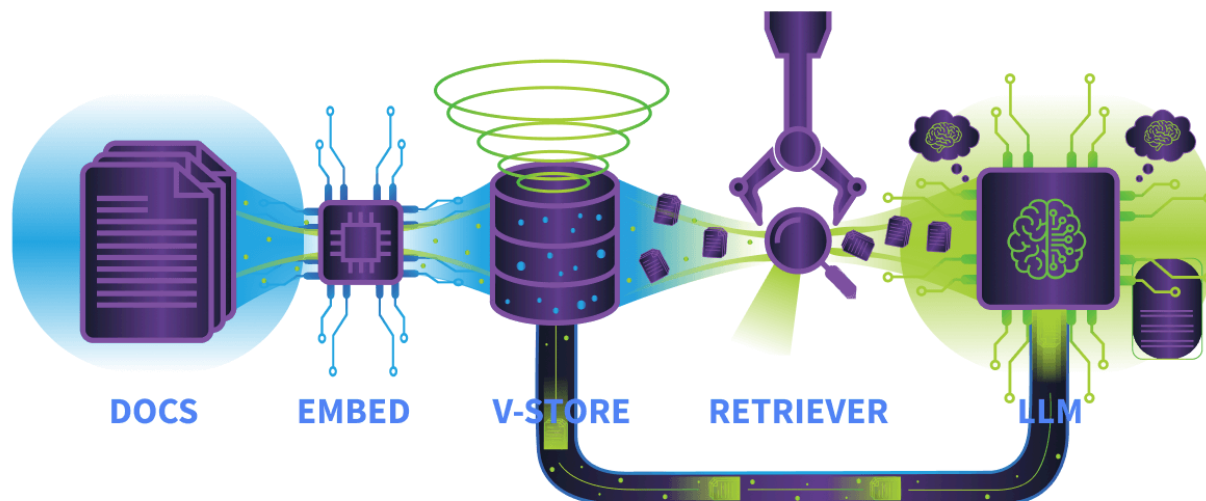
RAG (Retrieval-Augmented Generation) is a technique that enhances AI model responses by retrieving relevant external information at query time and feeding it into the model's context alongside the prompt.

Retrieval-Augmented Generation (RAG) allows language models to use external documents when answering questions. Instead of relying solely on training data, a RAG system retrieves relevant information at inference time and provides it as additional context before generating a response. This improves accuracy and freshness, but it also changes how trust works in the system. This room covers how RAG systems work, where their unique security risks appear, and how attackers exploit retrieval, context injection, and trust boundaries to manipulate model outputs.

Task 2 - RAG Architecture Overview

In a RAG deployment, the application provides retrieved content alongside user input. The model follows the structure of the input it receives, but it does not independently verify whether the retrieved information is correct, safe, or appropriate. This introduces specific security risks.

Because retrieval happens at inference time, malicious or misleading documents can influence responses without retraining the model. This is known as inference-time data poisoning. Retrieved content can also manipulate context by framing information in ways that alter the model's behaviour. In some cases, retrieved documents may contain instruction-like text that overrides system intent, even though the prompt itself was not modified. Securing a RAG system requires controlling how external data is selected, injected, and constrained during retrieval. To understand where these risks appear, you need to know how a RAG system is structured.



Core Components of a RAG System: A typical RAG system includes the following parts:

- **Embedding Model:** This model converts text into vectors; both user queries and documents go through this process.
- **Vector Store:** The vector store stores document embeddings; these embeddings represent the meaning of text as numerical values, enabling similarity comparisons.
- **Retriever:** The retriever is responsible for finding relevant documents based on a user's query, it uses similarity matching rather than exact keywords.
- **Language Model (LLM):** The language model generates the final response using the retrieved documents as context.

How Data Flows Through RAG: A simplified RAG workflow looks like this:

- The user submits a query
- The query is converted into an embedding
- The vector store searches for similar document embeddings
- Top matching documents are retrieved
- Retrieved content is injected into the LLM's context
- The LLM generates a response

At no point does the model verify whether the retrieved data is correct or safe.

Where Security Risks Concentrate: Although every component matters, risk concentrates in three areas:

- **Ingestion:** Malicious documents entering the system
- **Retrieval:** Poisoned documents ranking highly
- **Context injection:** Retrieved content influencing generation

Answer the questions below

What numerical representation is used to capture the meaning of text in RAG systems?

Embeddings

Which component selects the documents for the LLM?

Retriever

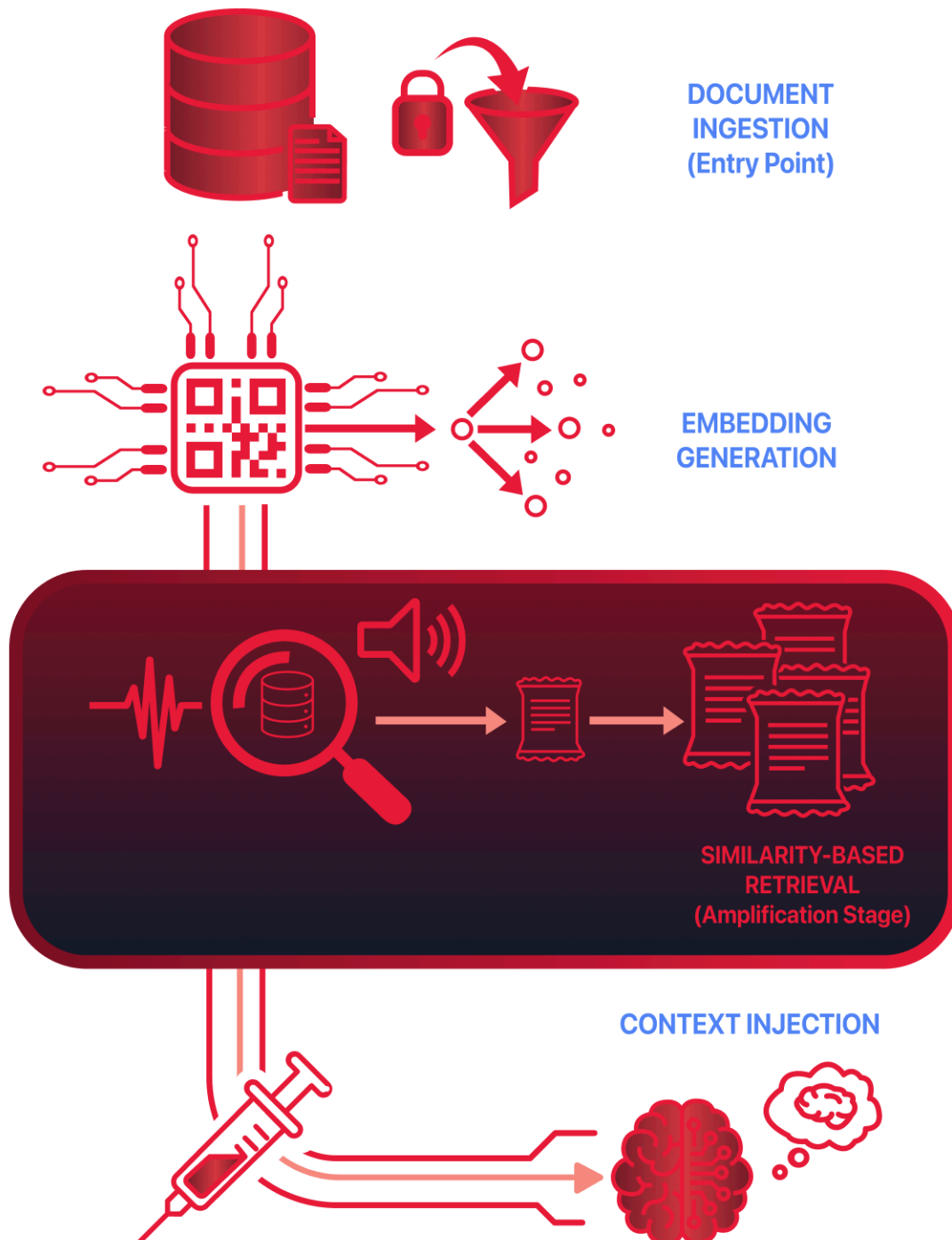
Task 3 - RAG-Specific Attack Surface

RAG systems introduce several new areas where security failures can occur. Unlike traditional applications, external data is not just stored — it directly influences model reasoning at inference time. The main RAG-specific attack surfaces are:

- **Document ingestion:** Untrusted, outdated, or malicious documents can enter the system if validation is weak.
- **Embedding generation:** Text is converted into numerical vectors, making intent and safety harder to inspect manually.
- **Similarity-based retrieval:** Documents are selected by semantic relevance, not correctness, trust, or safety.
- **Context injection:** Retrieved documents are injected directly into the model's context window before generation.

Each stage increases the system's exposure to manipulation.

RAG-SPECIFIC ATTACK SURFACES



1. Document Ingestion: RAG systems often ingest data from shared drives, wikis, or automated feeds. If validation is weak, untrusted or malicious documents can enter the knowledge base and become treated as trusted information.

2. Embedding Generation: Ingested documents are converted into embeddings. This process removes context such as authorship or approval status, making malicious and legitimate content appear equally valid.

3. Similarity-Based Retrieval: Documents are retrieved based on semantic similarity, not trust or intent. Attackers only need their content to “sound relevant” to influence retrieval results.

4. Context Injection: Retrieved documents are injected directly into the model’s prompt. The model cannot distinguish instructions from data, treating all retrieved content as trusted context.

Why Retrieval Is the Highest-Risk Component: Retrieval happens automatically and invisibly to the user. The language model:

- Cannot see where the documents came from
- Cannot verify document intent
- Cannot distinguish instructions from data

Once content is retrieved, it is treated as trusted background information. This makes retrieval one of the most security-critical components in a RAG system.

Answer the questions below

| Which RAG stage introduces the largest indirect attack surface?

Retrieval

| What component is lost during embedding generation that affects security?

Context

Task 4 - Retrieval Abuse & Context Manipulation

Retrieval abuse is a **RAG-specific attack technique in which unintended or malicious documents influence model output via retrieval**. This does not always require active manipulation by an attacker. In many cases, a malicious or

misleading document is already present in the knowledge base and is retrieved automatically during normal queries. Unlike traditional prompt injection, the attacker does not interact directly with the model's prompt. Instead, they influence **what data the retriever selects**.

Active vs Passive Retrieval Abuse: Retrieval abuse can occur in two common ways:

- **Passive poisoning:** Malicious content is ingested once and left in the knowledge base. The attacker waits for normal queries to retrieve it.
- **Active manipulation:** Content is deliberately crafted to rank highly for common or sensitive queries.

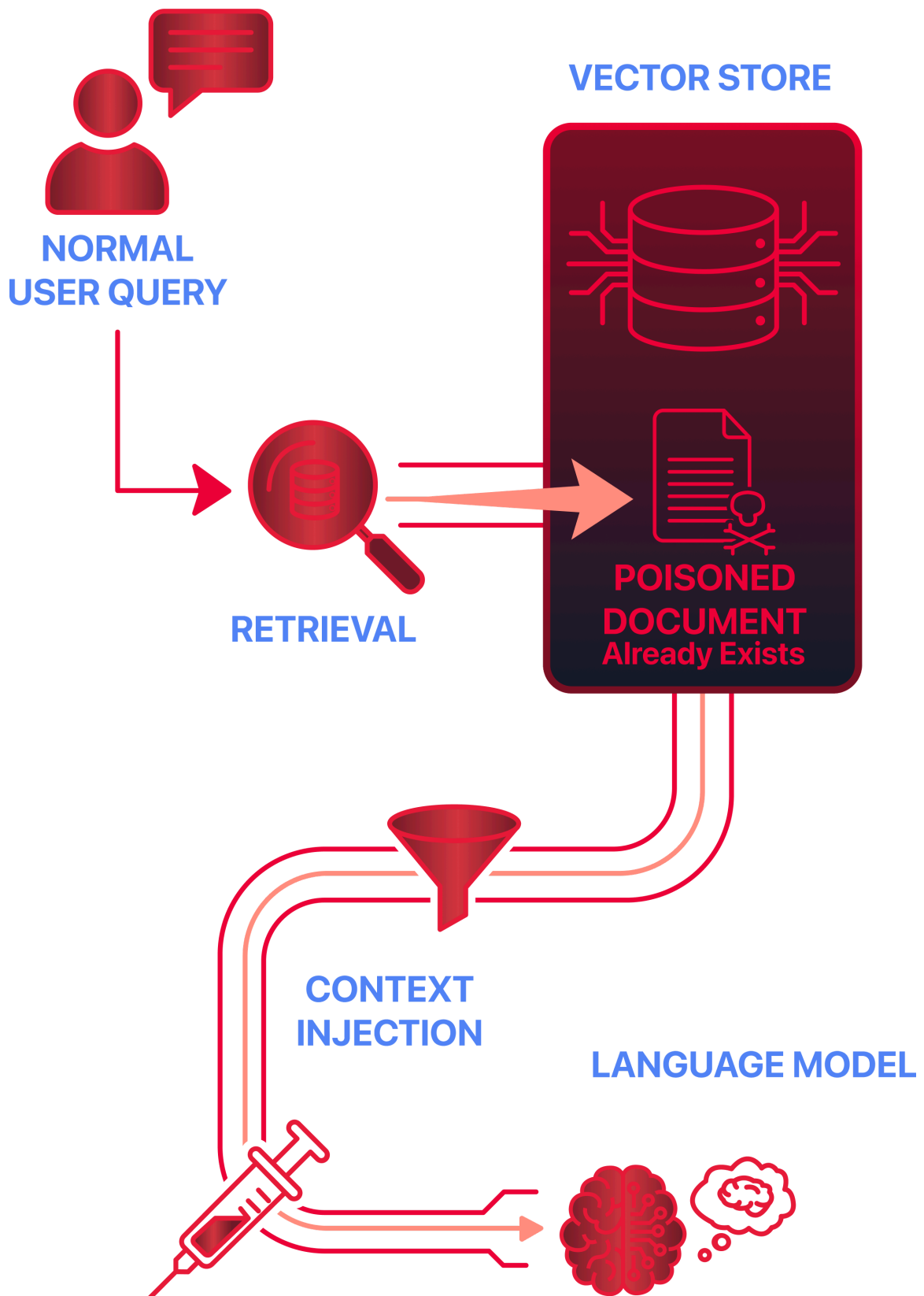
In both cases, the attacker does not need continuous access to the system.

How Context Manipulation Works: In RAG systems, retrieved documents are injected into the model's **context window** before generation. Problems arise when a retrieved document:

- Contains misleading or false information
- Includes hidden instructions framed as documentation

Retrieval selects documents based on **semantic relevance**, not intent or safety. If a document ranks highly, it is included in the context used for generation.

Why the Model Cannot Defend Itself



From the model's perspective, all retrieved content looks the same, because the model:

- Cannot verify document intent
- Cannot see retrieval rankings
- Cannot reliably distinguish instructions from data

Once content appears in the context window, it is treated as **authoritative input due to placement**, not because it has been verified. This is a design limitation, not a configuration mistake.

Why Retrieval Abuse Is Difficult to Detect: Retrieval abuse is difficult to spot because:

- Outputs may appear logical and well-structured
- No visible prompt injection is present
- Logs may only show "relevant documents retrieved."

From the system's point of view, retrieval is working as designed.

Security Impact of Context Manipulation: By manipulating retrieval, attackers can:

- Influence responses without modifying prompts
- Indirectly override system intent
- Introduce subtle misinformation or unsafe guidance

This is why retrieval must be treated as a **security boundary**, not just a performance feature.

Answer the questions below

What retrieval abuse technique involves crafting malicious content so it ranks highly for sensitive queries?

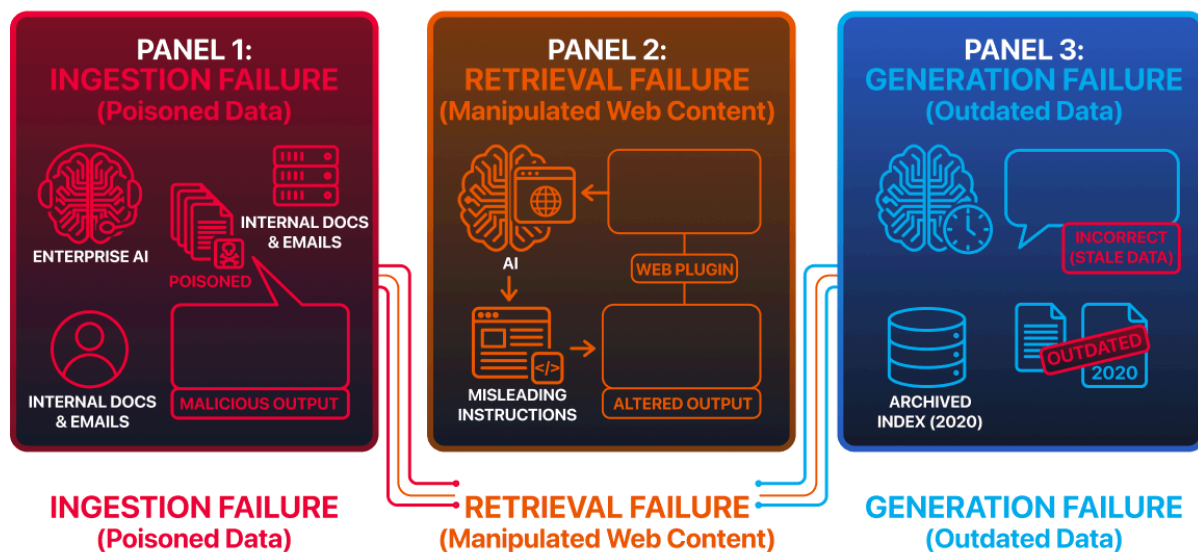
Active Manipulation

What does retrieval select documents based on?

Semantic Relevance

Task 5 - Real-World RAG Failure Scenarios

Failures in RAG systems are often subtle. In real deployments, responses may appear logical, well-written, and authoritative — while still being incorrect, unsafe, or misleading. These failures occur when **retrieved content influences the model's output in unintended ways**. The following case studies are based on **publicly documented incidents** involving deployed AI systems.



Case Study 1: Microsoft Copilot – Email-Based Retrieval Abuse (2026)

Microsoft 365 Copilot (opens in new tab) integrates directly with enterprise email, document, and calendar data to assist users with summarisation and question answering. In 2026, it was demonstrated that **content embedded inside emails** could be retrieved by Copilot during normal queries and influence its responses — despite not being part of the user's prompt.

How the Failure Occurred

- Emails were treated as valid ingestion sources
- Retrieved email content was injected into the model's context
- The model could not distinguish:
 - Legitimate information
 - Embedded instructions

- Misleading guidance

From the system's perspective, retrieval and generation worked correctly.

What Went Wrong

- Trust was implicitly granted at ingestion
- Retrieval surfaced unverified content
- Context injection amplified the impact across responses

Impact

- Sensitive enterprise information was exposed
- Organisations restricted Copilot access
- Microsoft issued security guidance and mitigations

This incident demonstrated how **internal data sources can still act as an attack vector when retrieval is not treated as a security boundary.**

Case Study 2: ChatGPT Plugins – Untrusted External Content (2023)

[ChatGPT plugins](#)[\(opens in new tab\)](#) enabled the model to retrieve live data from external services, including web pages and third-party APIs. In multiple cases, retrieved external content contained **instruction-like text** that influenced the model's behaviour once injected into the context window. This occurred without modifying the system prompt or model parameters.

How the Failure Occurred

- External sources were trusted by default
- Retrieved content was injected directly into context
- The model followed instructions embedded in retrieved data

This is a clear example of **indirect prompt injection via retrieval.**

What Went Wrong

- No validation of the retrieved content intent
- No separation between data and instructions
- Retrieval expanded the trust boundary beyond the application

Impact

- Unsafe or manipulated outputs
- Plugin features temporarily disabled
- Retrieval and plugin security models redesigned

Case Study 3: Web-Connected AI Assistants – Stale and Incorrect Retrieval

Several AI assistants that rely on indexed web content have returned **outdated or incorrect guidance**, even after the original sources were updated. These failures were not caused by attackers, but by governance gaps in retrieval pipelines.

How the Failure Occurred

- Documents remained indexed after changes
- The retrieval pipeline prioritised semantic relevance over freshness
- Outputs were presented as current and authoritative

What Went Wrong

- No document lifecycle management within the retrieval pipeline
- No freshness or version validation
- Retrieval amplified stale content across multiple queries

Impact

- Users followed incorrect guidance
- Operational and compliance risks increased
- Trust in AI systems was reduced

This shows that RAG failures do not require adversaries — poor governance in the retrieval pipeline alone is sufficient.

In these cases:

- Documents remained indexed after updates
- Retrieval prioritised relevance over freshness
- Users received outdated responses presented as current

What went wrong:

- No document lifecycle or freshness controls
- Retrieval amplified stale content
- Outputs appeared authoritative

Why These Failures Are Dangerous

RAG failures are risky because:

- Responses appear logical and well-written
- There is no visible prompt injection
- Users trust AI generated output

In many cases, the system behaves exactly as designed — but still causes harm.

Answer the questions below

In the Web-Connected AI Assistants cases, failures were caused by governance gaps in what specific part of the system?

Retrieval Pipeline

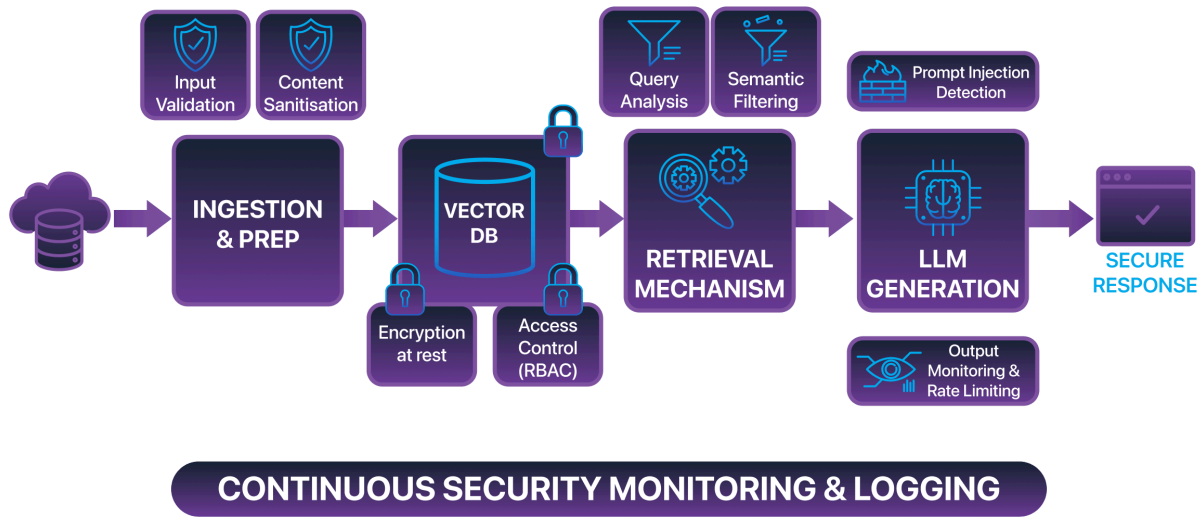
Task 6 - Defensive Considerations for RAG Systems

Detecting RAG abuse is difficult because poisoned or misleading content often looks legitimate. Retrieved documents:

- Are semantically similar to the query
- Blend with clean, approved content
- Produce outputs that appear logical and well-written

There is no single signal that reliably indicates abuse. In many cases, the system behaves exactly as designed while still producing harmful outcomes. As a result, detection often relies on observing how the system behaves over time rather than identifying a single malicious input.

Realistic Layered Defense Architecture (RAG) System



Guardrails on Retrieved Content: Guardrails aim to limit how retrieved content can influence the model. Common approaches include:

- Limiting how retrieved text is inserted into prompts
- Separating retrieved data from system instructions
- Applying heuristics to flag instruction-like patterns

However, these controls are **imperfect**. Instruction-like language is often ambiguous, and attackers can rephrase or obfuscate content to bypass simple checks. Guardrails reduce risk, but they do not guarantee safety.

Validation During Ingestion: Strong ingestion controls prevent many issues before retrieval occurs. Effective validation includes:

- Reviewing document sources
- Enforcing approval workflows
- Tracking ownership and update history

Once untrusted data enters the vector store, detection becomes significantly harder and more resource-intensive.

Monitoring and Output Review: Even with guardrails and validation, failures can still occur. Monitoring should focus on behavioural signals such as:

- Unusual retrieval patterns
- Repeated retrieval of the same documents
- Gradual changes in response tone or behaviour

These gradual changes are often referred to as output drift — a slow shift in how the system responds over time. Output drift is a key warning sign of poisoning, as it reflects gradual influence from malicious or misleading data rather than a sudden failure. Behavioural monitoring is often the most effective way to detect RAG poisoning, as it captures subtle, long-term deviations that other controls may miss. Regular review helps detect subtle, long-term influences that automated controls may miss.

Why Defence Must Be Layered: No single control fully protects a RAG system. Effective defence requires overlapping safeguards that:

- Reduce the likelihood of successful abuse
- Limit the impact of failures
- Detect problems early

RAG security depends on defence-in-depth, not on a single protective mechanism.

Answer the questions below

| What type of monitoring is a useful way to detect RAG poisoning?

Behavioural

| What does output drift reflect instead of a sudden failure?

Gradual Influence

Task 7 - Conclusion

Retrieval-Augmented Generation changes how trust operates in AI systems by allowing external data to influence model outputs at inference time. While this can improve relevance in some scenarios, it also introduces **new security risks when retrieved content is untrusted, manipulated, or poorly governed.**

Framework Perspective: The risks explored in this room align with how modern AI security frameworks model retrieval-driven failures.

- **OWASP Top 10 for LLM Applications**
 - **LLM01 – Indirect Prompt Injection:** Retrieved content can influence model behaviour without direct access to the prompt.
 - **LLM04 – Data & Model Poisoning:** Inference-time poisoning occurs when untrusted or stale data is retrieved and amplified.
 - **LLM07 – Insecure Model Monitoring:** RAG failures often remain undetected without retrieval and output monitoring.
- **NIST AI Risk Management Framework**
 - **Map:** Identify dependencies on internal and external knowledge sources.
 - **Measure:** Evaluate how retrieved data affects outputs.
 - **Manage:** Apply controls across ingestion, retrieval, and monitoring.
- **EU AI Act**
 - **Article 9:** Risk management for system behaviour.
 - **Article 10:** Data governance, quality, and lifecycle management.

Across all frameworks, retrieval risks are treated as system-level trust failures rather than model defects.